White paper

Firestorm: An Object-Oriented Data Access API and ODM tool for Firebase in Flutter Applications

Nicos Kasenides — nkasenides2@uclan.ac.uk[a]

[a]UCLan Cyprus

This document was compiled on June 4, 2026

Executive Summary

The increasing diversity and complexity of modern software applications has led developers to adopt flexible Backend-as-a-Service (BaaS) solutions like Google's Firebase. Despite providing several advantages, such datastores lack mature Object-Relational/Document Mapping (ORM/ODM) tools in cross-platform frameworks such as Flutter, which are particularly useful for bridging the impedance mismatch between application code and database structures in document-based systems. This white paper presents Firestorm, which is an object-oriented data access API (Application Programming Interface) and ODM tool for use with Firebase's Firestore and Realtime database. Firestorm aims to remove the *impedance mismatch* between object-oriented code data models used in application logic and the schemaless, document-based structure used in NoSQL datastores by providing type-safe, declarative mechanisms with which developers can define data entities, generate serialization and deserialization logic at compile-time, and utilize a unified set of data management operations under a common data access API. Firestorm was evaluated based on three aspects: measurements of its performance overheads, level of maintainability in its generated code, and its feature coverage compared to similar tools. The performance measurements revealed that Firestorm not only stays within an acceptable 10% performance overhead in terms of processing speed, but also surpasses code written using the native APIs by improving execution speed in several function calls. The maintainability of code written using Firestorm's data access API was measured using the Maintainability Index (MI). This was compared against code written using the native APIs, showing an average improvement in MI of $\approx 10\%$ across the sampled operations. Furthermore, Firestorm's feature coverage was compared against 20 other ODM and ORM tools based on 13 features, and showed that Firestorm has a very good level of feature coverage of approximately 77% and placing it among the most comprehensive tools of its type. Given the high adoption of cross-platform development frameworks like Flutter in recent years, Firestorm may lead to improved Developer Experience (DX), and promote modular, maintainable, and expansible software design.

Keywords: *Databases, Object-Relational Mapping, Backend-as-a-Service, Cross-Platform, Software, Firebase*

1. INTRODUCTION

1.1. Background and Context

Software applications have grown increasingly complex over the years. The rise of commercially accessible Artificial Intelligence (AI) and the Internet of Things (IoT) means that the services provided by such apps must often carry data intensive operations [1, 2]. There are a number of options to choose from to enable data persistence when developing software, and the choice usually depends on the software requirements for each product. Traditional database management involves the use of *Relational Database Management Systems* (RDBMS) such as MySQL, Microsoft SQL, or Oracle SQL, which are constructed based on the mathematical principles of sets and relations [3]. Such systems organize information using tables, rows/records, and columns/attributes, which have specific properties, such as strict schemas, typesystems, and leverage relationships to create dependencies between information [4]. The use of RDBMSs in various types of software enables complex queries through the use of SQL (Structured Query Language), a high level of data integrity, and support for various types of security constraints.

Despite their prominence, RDBM systems are not inherently scalable, which impacts their usefulness when it comes to managing large amounts of data such as those seen in Deep Learning (DL), or those used by context-sensing systems. For this reason, many systems choose to adopt NoSQL datastores which provide more flexibility by following a less rigid approach to structuring data. NoSQL datastores store information in key-value pairs, making them significantly easier to scale through *partitioning* and *sharding* [5–7]. This leads to more efficient scaling, and ultimately better performance especially under real-time constraints [8, 9]. For example, Firebase, a popular Backend-as-a-Service (BaaS) offering by Google, provides two types of NoSQL datastores: Firestore, and Realtime Database. Firestore falls under the category of document-based NoSQL datastores, whereas the Realtime Database falls under the category of key-value based datastores – akin to following a JSON structure [10]. Due to their increased flexibility and ease of use, such datastores are particularly suitable for cross-platform applications, such as those developed using the Flutter and React Native frameworks.

While there are plenty of advantages when using these datastores, lower adoption compared to RDBMSs means that there is a lack of support and tools, especially when it comes to combining and using these in cross-platform software. Cross-platform frameworks like Flutter and React Native are built using languages such as Dart and JavaScript, which are inherently object-oriented. This means that source code for this type of software, as with many other types of modern software, is centered around the object-oriented principles of inheritance, polymorphism, abstraction, and encapsulation [11]. Conversely, relational databases use tabular structures while NoSQL databases utilize key-value structures, which are significantly different from object-structured allocated memory. These types of data representations cannot be directly translated into memory/application objects that can be used in code directly, leading to an *impedance mismatch* between the two types of structures. While relational databases accommodate for this problem through the use of tried-and-tested *Object-Relational-Mapping* (ORM) techniques and tools, NoSQL datastores, and particularly those offered by Firebase, lack mature Object-Document Mapping (ODM) support, especially for cross-platform software written in Dart and Flutter. As a consequence, developers must write and maintain boilerplate code which enables the conversion to and from objects and key-value/document structures – increasing the maintenance cost of software, and introducing various types of bugs that are impervious to many hours of debugging. The result is poor code portability, limited reusability, and poor *Developer Experience* (DX) due to increasing overheads over time [12].

Some tools have been developed to solve this problem in different environments. For instance, the Java and Node.js ecosystems offer ODM tools like Morphia and Sequelize to convert to and from documents and memory objects. Interestingly, an effort is underway to create ODM tools for cross-platform, Flutter-based environments as well. The Firebase team has been developing the `firestore_odm` package, which "*provides generated Cloud Firestore bindings for Dart classes, allowing type-safe queries and updates*" [13]. While offering a significant step forward, this tool – like many other Flutter features – has seen relatively slow progress. The ever-present need for developers to efficiently manage data across cross-platform software means that this slow progress does not effectively capture the needs of developers for data management. Furthermore, this and various other ODM packages for Flutter applications have very poor documentation as witnessed in their example codes, and README documents, making them harder to understand and use. Finally, tools such as Firestore ODM reduce code maintainability by generating code which is not backward/forward compatible and does not enable the use of an extensible data access API.

Such problems can be solved by ***Firestorm***, a data access API (Application Programming Interface) and ODM tool for Flutter-based applications, which is specifically tailored towards the Firestore and Realtime Database. By leveraging various software engineering concepts such as model-driven design, it aims to solve the *impedance mismatch problem*, enabling developers to seamlessly manage data in these datastores without having to introduce additional logic in their software. As established, such a tool can be an important asset for maintainable software development. This builds on top of previous work which focused on building such an ODM tool for Java-based environments. While there are similarities between Java and Dart, the realization of such a tool in Dart entailed bridging various challenges, such as the lack of support for Reflection in Flutter apps, differences in language syntax and features, and variations in how languages support object-oriented design. Finally, a major challenge was the reduction of serialization/deserialization overheads, which are introduced during the document conversion process. Such overheads have been reduced in the Java-based environment by leveraging techniques including multithreading through Java's Executor Framework. Meanwhile, Dart uses different constructs such as concurrent execution through its event loop and Isolates mechanisms which provide support for parallel execution. Such challenges have been addressed in meeting the project objectives, which are enumerated below.

1.2. Objectives

The first objective was to identify and study relevant research works and similar tools which have been developed in the past, with the aim of uncovering key challenges and gaps, as well as identifying key features of ORM/ODM tools. Once these were studied, Firestorm was designed, implemented, and tested as a Flutter package. Finally, the tool was evaluated with respect to its performance, code maintainability, and feature coverage, ultimately leading to conclusions regarding Firestorm's usefulness as a development tool, but also to discover its limitations for future improvements.

Firebase's set of tools, including the Firestore and Realtime Database, are among the most popular options when it comes to adding backend support to cross-platform software. Such cross-platform products are used in various contexts and domains, including educational, commercial, and research projects. Yet, despite their prominence, the Dart/Flutter developer community lacks a reliable, developer-friendly ODM solution to support development in both the Firestore and Realtime database. Based on this, the objectives of this tool are to:

1. Provide a consistent, declarative approach to model, access, and manipulate data by leveraging model-driven development practices.
2. Reduce or eliminate repetitive boilerplate code to manage data.
3. Facilitate a Rapid Application Development (RAD) approach by accelerating the development of software prototypes.
4. Improve code readability, maintainability, and portability, especially in larger-scale Flutter projects.
5. Support both datastores using a consistent data access API.

1.3. The advantages of adopting Firestorm

The adoption of Firestorm provides a variety of technical advantages for Flutter applications, as shown in Figure 1. Such a tool may also provide a way to support research and teaching in software engineering by encouraging the use of object-oriented design, data abstraction practices, and the efficient use and management of data in applications requiring scalability, such as those frequently utilized in Machine Learning and Data Analytics.

Figure 1. Several advantages an application's developers and users can enjoy if Firestorm is adopted.

1.4. Overview

The rest of this paper is organized in the following sections. Section 2 explores existing relevant tools, software architectures, the challenges faced, and their impact on software development. Section 3 specifies the project's requirements, the Firestorm architecture, and describes development details. Section 4 outlines the testing strategy and testing practices followed and is followed by section 5 which presents and discusses the results of the evaluation. Finally, section 6 concludes the paper by reflecting on the project's contributions and impact in the software development landscape.

2. EXISTING SOLUTIONS, LIMITATIONS, AND MOTIVATIONS

2.1. Introduction

The NoSQL database paradigm has shifted how we develop modern web and mobile applications, and provides a versatile alternative to the rigidity of traditional relational database systems. Given today's needs for scalable services, the use of datastores such as Firestore, and Realtime Database becomes increasingly important. Such datastores have undoubtedly influenced the software architecture and development practices used in modern software development. This section focuses on a discussion of various types of NoSQL databases and their characteristics, as well as existing tools offering ORM/ODM for this specific domain.

2.2. NoSQL Datastores

NoSQL datastores can be classified into four different categories [14]:

1. **Key-value datastores** – Redis, AmazonDB, Realtime Database, etc. which store data by formating pairs of keys and values. Typically enjoy strong scalability but provide less support for complex queries [15].
2. **Document datastores** – MongoDB, Firestore, etc. which store data in documents using JSON-like structures. This allows more complex structures like nesting, collections of documents, and more slightly more powerful querying systems.
3. **Column-Family datastores** – Apache Cassandra, HBase, etc. extend the key-value approach by utilizing column families and rows and are particularly useful for applications which induce write-heavy workloads and time-sorted data [16]. Requires deep understanding of various concepts to master.
4. **Graph datastores** – Neo4J, Amazon Neptune etc. can enable the storage of data that is strongly interconnected and consists of *nodes* and *edges* with their associated data, which enables the execution of complex graph-based queries [17]. They offer limited scalability and introduce a steep learning curve for software developers [18].

2.2.1. *Other characteristics of NoSQL datastores*

A notable feature of various types of NoSQL datastores is support for *eventual consistency*. In contrast with the strong consistency employed by relational systems that comply with the *ACID principles* (Atomicity, Consistency, Isolation, Durability), eventually consistent databases can provide advantages in performance, scalability, and data availability. Such systems focus on meeting the availability and partition tolerance requirements for two of the properties of the *CAP theorem* (Consistency, Availability, Partition Tolerance) in sacrifice of strong consistency [19]. Another characteristic of NoSQL datastores is that in many cases there is a need to perform client-side queries/joins as there is poor support for database-side queries. This can be particularly expensive both in terms of cost but also in terms of resource utilization, especially when large amounts of data must be extracted. Finally, performance enhancing techniques such as normalization become more obscure, as these are not as well-defined as in their RDBMS counterparts [20], which can impact code complexity and maintainability.

2.3. Firebase

Firebase, initially developed by Firebase Inc. and later acquired by Google, is perhaps one of the most known Backend-as-a-Service (BaaS) platforms that support software development. Firebase offers two cloud-hosted NoSQL databases called Firestore and Realtime database. These databases are popular options for developers when it comes to realizing application data persistence on the cloud. They offer document/key-value storage, features such as real-time data synchronization, offline capabilities, and integration with security policies and authentication services. This high level of integration with other services, makes these databases suitable for rapid prototyping, and especially attractive for cross-platform application development [21].

Firebase's networking architecture is significantly different from the traditional client-server model. Rather than employing a unidirectional data flow, Firebase utilizes RESTful APIs over HTTP to open persistent WebSocket connections that enable real-time bidirectional traffic to and from client devices. This approach significantly improves performance by decreasing latency but also simplifies application logic — especially for collaborative or real-time applications where data must be synchronized quickly. While these are major advantages, Firebase introduces some considerations regarding data security, caching policies, and conflict resolution [21].

Studies evaluating Firebase's scalability and performance have revealed overall positive results but have also identified bottlenecks when retrieving nested data and while experiencing write surges in within large datasets [22]. Firebase has also been evaluated in the context of real-time distributed systems such as MMOG backends, exploring how such databases can cope with the performance and scalability requirements in update-heavy environments [23]. Furthermore, the cost implications of using Firebase databases and the vendor lock-in risks associated with Firebase and similar platforms have also been documented [24].

2.4. ODM tools

The gap between object-oriented programming design and database structures is known as *impedance mismatch* [25]. To bridge this gap, developers employ various Object-Document Mapping (ODM) frameworks which automate the serialization and deserialization to and from in-memory objects to persistent data in the database (e.g., documents). Examples of such ODM tools are Mongoose for MongoDB and Morphia, but also some community-developed Firebase ODM tools such as firestore_odm and Typesaurus.

ODM tools can provide a variety of features. For instance, Mongoose provides schema validation mechanisms and query-building APIs that help to abstract the underlying data structure. Such abstractions can improve code maintainability and reduce the code needed to manage data [26]. When using Firebase, developers rely on libraries which integrate with the Firestore or Realtime Database SDK, but such tools frequently lack standardized schema enforcement. Typesaurus, a JavaScript library, attempts to bridge this gap by offering schema and type safety for Firestore interactions, while the firestore_odm package for Flutter/Dart attempts to offer serialization/deserialization support, but lacks support for object-oriented transactions and inheritance in its data models [21]. Some ODM tools also provide more advanced features such as tracking changes, concurrency controls, and lifeycle event hooks. Such capabilities may align with developer expectations from ORM tools used in relational database, but can also conflict with the eventual consistency and schema flexibility approach used in NoSQL databases [18].

2.5. The challenges of using ODM tools

ODM tools can face challenges in terms of code maintainability, scalability, and performance.

For instance, schema-less designs employed in NoSQL datastores tend to increase the likelihood of data inconsistencies and misrepresentations, and typically require additional validation on the client-side [20]. From a maintainability standpoint, this means that application code can become increasingly harder to evolve over time as developers start making implicit assumptions about the data and these spread across the code [18].

The primary motivation for adopting a NoSQL datastore is scalability. However, realizing horizontal scalability does not just depend on which datastore is used, but also on careful data modelling, using partitioning strategies, and understanding the trade-offs between

availability and consistency [23, 27]. For example, Firebase inherently provides horizontal scaling for data, but also recommends to developers to avoid excessively nested structures and query hotspots which can degrade performance under scale [21]. Therefore, ODM tools must be configured in a way such that they avoid these problems and can enable applications to take full advantage of NoSQL datastore horizontal scaling without detriments to scalability.

The performance of a system is another key consideration that can complicate design choices. Abstractions introduced by ODM trools can introduce overheads that impact processing time and latency, especially during serialization and deserialization and query translation. This is particularly important when handling large volumes of data or running complex queries [26], but also while synchronizing under heavy concurrent access [22, 28].

2.6. Potential impact on software development

The adoption of Agile methodologies has favored the use of rapid prototyping (RAD) and continuous deployment (CI/CD) of systems. Developers have utilized the flexible schemas of NoSQL datastores and the features of BaaS platforms, which align well with these objectives [14]. Consequently, the real-time capabilities of Firebase's datastores have accelerated the development of many collaborative and real-time applications in various domains such as education, gaming, and research [21, 23, 29, 30].

However, the cost of this agility can materialize as increased technical debt, if development teams fail to establish standards for robustly validating data. The lack of mature ODM frameworks more generally, but more specifically for the Dart/Flutter development environment and Firebase forces developers to implement custom logic for mapping objects to documents and vice versa, for each individual class in each individual app. This significantly hampers maintainability and increases the likelihood of introducing bugs that may go undetected for large periods of time [18]. Ultimately, the use of a tool like Firestorm may minimize some of these issues, ultimately leading to a better DX, improved focus on application features, and reduced time to market.

3. FIRESTORM'S ARCHITECTURE

3.1. Introduction

This section delves into the decisions taken to divide the package into layers and components using modular design principles, with all of these layers forming a unified software architecture [31]. The design of the data access API is discussed, along with the rules established regarding data definition and organization. The design section also presents code design decisions based on various elements of the Dart programming language. The overall design objective of this project is to provide an extensible, modular data access API that can offer high learnability and code maintainability in par with the requirements which are outlined next.

3.2. Requirements specification

The requirements were identified through the combination of related works in the academic domain, as well as an analysis of existing tools and their limitations mentioned in Section 2. Each of these requirements is categorized as functional or non-functional under various thematic groups.

3.2.1. Functional requirements

1. **Data Modelling and Validation**
   - Define data models using annotated Dart classes.
   - Perform static validation of data models.
   - Verify conformance to the Firestorm data model.
   - Support inheritance and perhaps composition.
   - Support enumerated types.
   - Register classes for use with the Firestorm API.
2. **Code Generation**
   - Generate serialization and deserialization methods.
   - Invoke generated mapping logic transparently without developer intervention.
   - Generate identifiers for database entities when required.
3. **Unified Datastore Access**
   - Support both Firestore and Realtime Database through a common API.
   - Provide support for offline-first access through a local file-based database.
   - Provide an object-oriented abstraction over datastore operations.
   - Enable seamless transition between supported datastores.
4. **Database Operations**
   - Support CRUD operations for both datastores.
   - Support queries and filtering.
   - Support batch operations.
   - Support Firestore transactions.
   - Provide error handling mechanisms.
   - Provide asynchronous and non-blocking APIs.
5. **Real-Time Features**
   - Provide object-oriented real-time listeners for Firestore.
   - Provide object-oriented real-time listeners for Realtime Database.
6. **Interoperability**
   - Allow access to underlying Firebase APIs and datastore references.
   - Maintain compatibility with existing Firebase SDK functionality.
7. **Configuration and Resource Management**
   - Configure datastore caching.
   - Support online and offline operation.
   - Manage datastore connections and resource deallocation.
8. **Advanced Features**
   - Support datastore subcollections.
   - Provide pagination APIs.
   - Utilize concurrency where appropriate to improve performance.
   - Support datastore emulators.
   - Allow customization of collection names.
   - Support Firestore sentinel operations.

3.2.2. Non-functional Requirements

1. **Maintainability and Developer Experience**
   - Reduce boilerplate code associated with serialization and deserialization.
   - Improve code maintainability compared to native Firebase APIs.
   - Provide comprehensive documentation for the framework and generated code.
   - Follow established Dart and Flutter conventions to maximize developer adoption.
2. **Performance and Reliability**
   - Maintain low performance overhead during datastore operations.
   - Ensure that data mapping operations do not corrupt datastore contents.

- Preserve the graceful degradation characteristics of the underlying Firebase SDKs.

3. **Platform Compatibility**
   - Support Flutter Web deployments.
   - Support mobile platforms, including Android and iOS.
   - Support desktop platforms, including Windows and macOS.
4. **Security and Interoperability**
   - Maintain compatibility with Firestore and Realtime Database security rules.
   - Remain interoperable with the latest stable Firebase SDKs.
5. **Extensibility**
   - Provide an extensible architecture capable of supporting future datastore integrations and framework enhancements.

3.3. Tools & Technologies

This project relied on the use of certain tools and technologies. Firstly, Dart and Flutter SDKs were used as part of the development environment. Secondly, the `firebase_core`, `firebase_database`, and `cloud_firestore` packages for the Realtime database and Firestore were needed, as they serve as the foundation of the package onto which the ODM layer was constructed. For code generation the `build_runner`, `source_gen`, `analyzer`, and `build` packages were instrumental, as they provided the necessary functionality to scan and analyze code, and to generate various functions related to serialization/deserialization at compile time. Git and GitHub were used for version control and the entire project is open-source and **available online**[1]. The development environment consisted of Android Studio, the Android SDK and Emulator which served as a testing platform, as well as the Java SDK and Windows 10 SDK which were needed to run and test Android-based and Windows-based applications. For testing purposes, Flutter's `flutter_test` and `integration_test` packages along with Firebase's emulator suite can be used to conduct testing by running a Flutter app as part of an Android ecosystem, which interacts with locally-hosted Firestore and Realtime database emulators.

3.3.1. Installing and using Firestorm

The `pub` command line tool and pub.dev – the official Dart and Flutter package repository – were used to deploy the package which is **now available for general use**[2] in Flutter applications by running `flutter pub add firestorm` in Flutter projects.

3.4. Design overview

Figure 2 demonstrates the intended setup and usage workflow for Firestorm to be used in a Flutter project. The diagram divides this process into two phases: setup and usage. The setup process (blue) involves one-off steps in which developers are intended to configure their project to use Firebase, whereas the usage process (green) is an iterative process where developers are expected to re-run several steps when applying data model updates to their applications.

The following checklist provides an explanation of these steps:

1. **Create Flutter project and install Firestorm:** The workflow begins by creating a Flutter project, where Firestorm is imported as a dependency through the Dart package manager (pub). This can be achieved by running the command `flutter pub add firestorm` on the project terminal.
2. **Firebase configuration:** The second step involves the generation of a Firebase configuration file (`google-services.json`) or a Firebase project configuration using the **FlutterFire CLI** – assuming an existing Firebase project – to create a connection to Firebase's services and provide access to Firestore and the Realtime Database.
3. **Define data classes:** Developers must then define their data entities. These are expected to be Dart classes annotated with `@FirestormObject()`, which will allow code analysis tools to detect and analyze these entities.
4. **Model generation:** The developer must run the build process using `dart run build_runner build –delete-conflicting-outputs` to analyze the entity classes and generate various pieces of code for serialization, deserialization, class identification, and more. The result of this process will be a single, generated file called `firestorm_models.dart` which contains adapter code.
5. **Import data models:** The `firestorm_models.dart` file needs to be imported by the developer to access critical functions needed to operate Firestorm. This can be done only once, in the file containing the `main()` function) to register the classes.
6. **Firestorm initialization:** The developer must initialize Firestorm within their code, depending on the datastore(s) they would like to use. Data entities will also eed to be registered with Firestorm so they can be used in operations, which can be done by calling the `registerClasses()` function within the main method as shown in Listing 1.

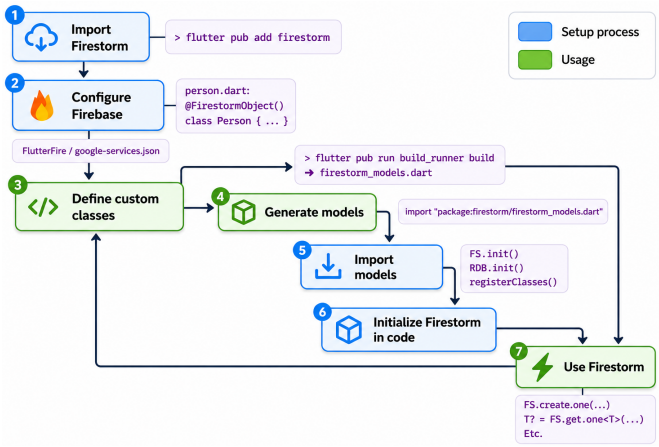

Figure 2. Firestorm setup and usage workflow

```dart
import 'package:firebase_core/firebase_core.dart';
import 'package:firestorm/firestorm.dart';
import 'package:<APPNAME>/generated/firestorm_models.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await FS.init(); //Init Firestore
  await RDB.init(); //Init Realtime DB
  registerClasses(); //Register classes

  runApp(MyApp()); //Run app
}
```

Code 1. Setting up Firestorm in the main function of a Flutter app.

7. **Use the Firestorm APIs:** Developers can utilize Firestorm to carry object-oriented data operations using its APIs, such as creating and reading objects to and from the database.

3.5. Software Architecture components

The software architecture focuses on a layered approach that aims to separate concerns across multiple high-level layers within the package. This improves the modularity of the package, facilitates the continuous integration of new features, and makes testing easier. The package is divided into high-level software components, which are shown in 3.

[1] **Firestorm for Dart and Flutter on GitHub (https://github.com/RayLabz/Firestorm/tree/master/firestorm_flutter)**
[2] **Firestorm on pub.dev**

Figure 3. The Firestorm Architecture and its components

This software architecture ensures that various software components such as code generators and runtime logic remain as decoupled as possible, which can improve the maintainability of the package itself and enable isolated unit/integration testing.

3.6. Data modeling

3.6.1. Data types

Since Firestorm needs to support both the Firestore and Realtime database, it needs to conform to the corresponding data types for each of these datastores in the Dart programming language. Types which are not supported by each of these datastores should not be accepted into the corresponding serialization and deserialization mechanisms, and therefore, the tools which generate code must take this into consideration. To meet the data type constraints of these two datastores, the static checking components must therefore differentiate the validation of data classes, and accept them for use by each datastore based on the accepted data types of each datastore.

Firestorm supports the native data types provided by both Firestore and Realtime Database while exposing them through a consistent object-oriented programming model. Both datastores support primitive Dart types including `String`, `int`, `double`, `bool`, and `null`, as well as collection types such as `List` and `Map<String, dynamic>`. Firestore additionally supports specific types including `Timestamp`, `GeoPoint`, `DocumentReference`, and binary data represented using `Uint8List`. The Realtime Database does not provide native support for these types and therefore relies primarily on primitive values and JSON-compatible collection structures. Firestorm abstracts these differences through its serialization and deserialization layer, enabling developers to work with a unified set of Dart objects regardless of the underlying datastore.

3.6.2. Data organization and constraints

To create a consistent data structure within each respective database, data entities should follow some organization principles and constraints. Firstly, user-defined classes should be annotated with the `FirestormObject()` annotation which will enable code analyzer tools to detect these classes and perform static checking and code generation. Furthermore, both datastores utilize String-based document and object IDs to identify items within the database. Therefore, each object defined for use with Firestorm must have a String-based `id` attribute, which serves this purpose. Documents and objects with the same ID are considered the same item, and so each object must carry a unique ID. To help generate unique, random IDs, Firestorm provides the method `Firestorm.randomID()`, which generates a UUID v8.

Firestorm handles the structuring of data internally by maximizing its potential for horizontal scalability. This is achieved through data *flattening*, which splits objects into different paths using *denormalization*. This means that data can be retrieved more efficiently in separate calls when it is needed, which can significantly increase horizontal scalability. Considering an example, if a database needs to store data related to chatrooms and messages that are related to these chatrooms, it can use a nested structure where chatroom messages are nested within each chatroom object. However, this means that when a query is made to retrieve information about the chatroom, messages for that chatroom will be retrieved as well, potentially causing significantly larger data reads and slower response times. For the Realtime database, which bills users based on data traffic rather than number of operations, this can also incur significantly higher costs. Thus, Firestorm's use of the flattened structure, where objects are split into multiple paths is cost-beneficial in the longer term. For example, chatroom information and messages could be stored independently, so messages can only be retrieved when needed to avoid these problems. This strategy can also be advantageous in Firestore, even though Firestore bills users per number of operations rather than data traffic. This is because Firestore uses a collection and document/subcollection structure, where data can be stored either as documents within collections, or as documents within subcollections of other documents. This approach allows queries to retrieve only the document data, and optionally retrieve subcollection data when needed, thus enabling more efficient operations. Firestorm follows the flattened data organization approach by mapping each class to a Firestore collection or root tree node in the Realtime database. Each object of that class will be stored under that collection, with the ability to create subcollections in both datastores to store objects in different groups when needed.

Another requirement is handling support for object-oriented concepts such as inheritance and composition, which may allow developers to freely create complex data hierarchies. To support inheritance, entity analysis must detect whether an analyzed class is a derived class, since data from its base class will also need to be utilized in the derived class's data model. These object-oriented concepts are considered important as they enable polymorphism, where developers may choose to retrieve an item of a derived class from the database, but represent it as its superclass object within their code whenever needed – enabling more deeply nested and complex class hierarchies that can better serve the technical demands of modern applications.

3.7. Validation and generation

3.7.1. Entity validation

An important step before generating code is to ensure that data entities defined by developers follow the constraints defined above. This ensures that database operations will be successful when they are executed and will not yield any errors related to invalid data types. This stage entails the use of static analysis to analyze code in defined classes. The analyzing components will first detect annotated classes. Classes which are not annotated will not be checked, and thus be ignored. If such classes are used in Firestorm operations through the data access API, Firestorm will yield an exception, warning the developer that this is an invalid class. The next validation stage is to check whether all annotated classes have a string-based ID fields, and whether they are composed only of valid data types through their class hierarchy.

3.7.2. Code generation

The code generation process involves four steps, which result to the creation of the `firestorm_models.dart` file.

1. Header generation
2. Import generation
3. Extension generation
4. Registry function generation

During the header generation step, the generator will create a header comment at the beginning of the generated file which will warn developers that this code is generated and should not be modified. The header also contains other information such as the generation date and tool version.

While the code analyzer checks entities during entity validation, the asset ID of each entity can be stored. The asset ID can be used to identify an asset (i.e., a code element such as a class) within the package in which it was defined. This is important for code generation, as it can enable generated code to import elements from existing files — such as user-defined classes. During the *import generation* step, the generator will use these asset IDs to generate Dart `import` statements which will allow the code to use various elements that are needed for the code to function.

The extension generation step will facilitate the creation of Dart extensions, which add functionality to existing user-defined data classes. In this case, this can be applied to the data entities defined by the user in order to extend their functionality with additional methods. The purpose of these methods will primarily be for serialization and deserialization, but extends to the management of classes and other metadata. Important methods such as `toMap()` and `fromMap()` are generated within each class extension, allowing objects of each valid data entity to be serialized to a Map data structure or deserialized to an object of that type. This step is important within the serialization mechanism, as both Firestore and the Realtime database receive and return data using Map data structures.

In the final step of the generation process, the code generator creates data structures and functions which are needed to register classes. An important function in this case is the `registerClasses()`, which automatically registers all valid classes with Firestorm after it is initialized. Other generated data structures may include function registries, which will contain generated entries mapping each data entity to its `toMap()` and `fromMap()` methods. These are necessary, as such methods may not be visible statically through the data access API, which works with `dynamic` objects. As such, these structures can be used to determine the correct functions to call when an object has to be either serialized or deserialized at runtime. These concepts stem from the Registry design pattern, which enables the use of a global registry to manage and provide access to a shared pool of objects, services, or configurations [32]. This pattern is coupled with the Strategy pattern to create pluggable components which are not discoverable until the code is compiled, thus enabling the use of the mentioned data structures and functions within application-specific code.

3.7.3. Data access API

The data access API provided by Firestorm is perhaps one of the most critical components of the package. The API must use a clear, easily learnable, and maintainable design which can enable developers to utilize it effectively, and to maintain and improve the package in later stages. 8 key principles are guiding the API's design, and shown in Figure 4: *Consistency*, *Extensibility*, *Type safety*, *Performance*, *Error prevention*, *Declarative approach*, *Error specificity*, and *Testability*.

Figure 5 shows an overview of the data access API. Several design decisions were considered while designing this API. Firstly, it is divided into three main components. The `Firestorm` class is designed to hold common functions used by both Firestore and Realtime database.

Figure 4. The principles guiding the design of the Firestorm API.

Subsequently the `FS` and `RDB` classes serve as *facades* and are responsible for exposing code which handles operations for each of the two datastores. These classes should not be derived from a parent class even though they have common operations, as this could limit the potential expansion of the package to support more datastores in the future. The lack of Dart support for nested static classes (such as those supported by Java) necessitates the use of such *facades* to enable a *namespaced* design.

Each of the two classes contains configuration and management methods, as well as the *data access delegates*. The data access delegates are classes which host functionality related to each type of datastore operation and can be accessed through properties defined within the `FS` and `RDB` classes respectively (e.g., `create`, `get`, etc.). For instance, the `create_delegate` object could enable developers to access its methods to create one or multiple objects within the database. This can result to a one-liner statement similar to `FS.create.one()`, with an object to be created being passed to the function to create a single document in the database. Another important reason for this type of API design is the fact that Dart does not support function overloading as it is a dynamically typed language. As a consequence, it is not possible to use the same function name to conduct the same type of operation (e.g., using `FS.create()` to pass both a single object and multiple objects). This is a significant difference from the Java-based Firestorm API, which utilized function overloading and varargs parameters to support such design. Hence, these language pecularities necessitated the use of the Delegate pattern to ensure that multiple types of operations are accessible from a single place, are easily discoverable, and with concerns separated to individual classes rather than a single class.

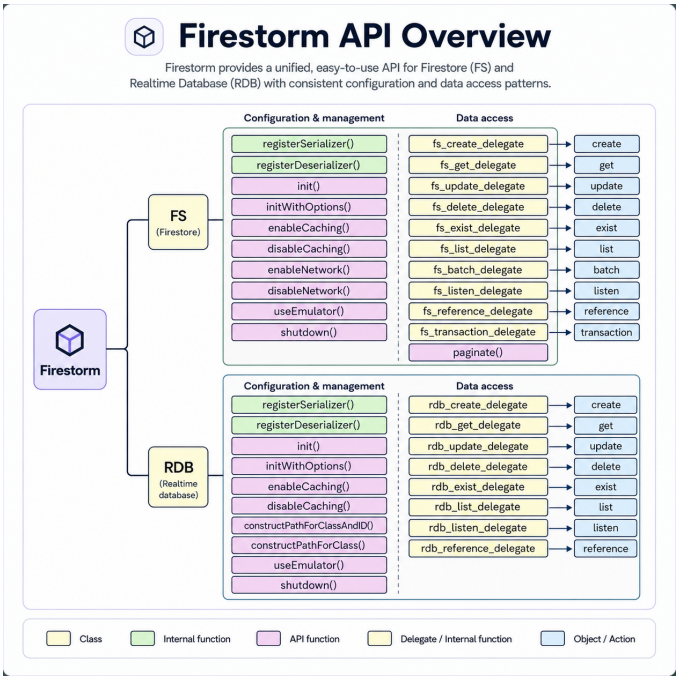

Figure 5. An overview of the data access API, including its classes, functions, and objects.

4. TESTING

4.1. Testing strategy and platform

Firestorm was tested to (a) verify the correctness of the code analysis and generation components, (b) confirm the correctness of data operations in both datastores, and (c) ensure the reliability of real-time data updates. Testing was undertaken by creating and utilizing a Flutter project as the testing platform. The testing platform incorporated manually executed unit tests, along with integration tests using the `integration_test` package. These were imported to provide the necessary tools to run test code. The integration testing code is located under the `integration_test` folder and is broken into different files based on the operations and tools being tested. Android Studio and the Android Emulator were used to carry out testing on top of an emulated Android operating system. The Firebase Emulator suite was also used to conduct testing for each respective datastore, which was configured by creating rules and other configuration options for Firestore and Realtime database.

4.2. Unit Tests

Despite making several attempts to conduct automated unit testing on the entity analysis and code generation tools using `flutter_test` and `mockito`, this was not possible due to the compile-time nature of these tasks. Such testing automation tools are not compatible with tools running at compile time, such as the `analyzer` package which is responsible to provide class inspection features for entity analysis. As a result, unit testing was done manually by first defining unit tests, running them, and then comparing their expected and computed outputs. The following list shows a curated set of predefined unit test cases which were used during this testing process:

- `containsValidIdField()`
  - 1a: Class does not contain an ID field → Invalid class, no code generated.
  - 1b: Class contains an ID field of a type other than `String` → Invalid class, no code generated.
  - 1c: Class contains a valid `String` ID field → Valid class, code generated.
  - 1d: Class inherits a valid `String` ID field from a superclass → Valid class, code generated.
  - 1e: Both the class and its superclass define an ID field → Compile-time error.
- `validateAttributeTypes()`
  - 2a: Class contains only Dart core types → Valid class, code generated.
  - 2b: Class contains Dart core and supported Firestore SDK types → Valid class, code generated.
  - 2c: Class contains Dart core types, Firestore SDK types, and supported composite types → Valid class, code generated.
  - 2d: Class contains Dart core types, Firestore SDK types, composite types, and enum types → Valid class, code generated.
  - 2e: Class contains an unsupported composite type → Invalid class, no code generated.
- `generateExtensionMethods()`
  - 3a: Class uses a constructor with positional parameters only → Extension methods generated successfully.
  - 3b: Class uses a constructor with positional and named parameters → Extension methods generated successfully.
  - 3c: Class uses a constructor with positional, named, and optional parameters → Extension methods generated successfully.
  - 3d: Class uses a constructor with positional and optional parameters → Extension methods generated successfully.

The testing process revealed several problems, such as inconsistencies during the code generation process which resulted in code which did not compile in multiple stages of the project. In several cases, generated code which did compile resulted in unexpected null values due to incorrect code generation for mapping object data to map entries and vice versa, especially when it came to composite objects. The testing process also revealed the need to offer support for named parameters as more advanced versions of entities were introduced. In this case, the initial version of the analyzer and generator code did not detect named parameters at all, which resulted in empty mappings when an entity constructor only contained named parameters.

4.3. Integration Tests

Unit testing primarily aimed to verify the behavior of compile-time stages such as entity analysis and code generation. However, this testing approach is not helpful in covering any runtime functionality associated with datastore operations. To verify runtime functionality, integration tests were necessary as the Firestore and Realtime database SDKs are not compatible with unit testing in Flutter. The primary objective of integration testing is to verify the behavior of data operations across both Firestore and Realtime database, including basic operations, real-time listeners, batch writes, transactions, and pagination. Listings 2 and 3 show samples of code testing pagination and transaction operations for Firestore, while Listing 4 demonstrates the use and testing of the `RDB.list.ofClass()` operation for the Realtime Database.

```
testWidgets("Test paginate()", (tester) async {
    await FS.delete.all(Person, iAmSure: true); //Setup
    await FS.create.many(people); //Setup
    var paginator = FS.paginate<Person>(limit: 3);
    var currentPage = await paginator.next();
    assert(currentPage.items.length <= 3);
    while (currentPage.items.isNotEmpty) {
      currentPage = await paginator.next();
      assert(currentPage.items.length <= 3);
    }
});
```

Code 2. An integration test, testing the behavior of `FS.paginate()`.

```
testWidgets("Transaction test", (tester) async {
    await FS.delete.all(ComputingStudent, iAmSure: true);
    await FS.delete.one(student);
    await FS.create.one(student);
    FS.transaction.run((transaction) async {
      ComputingStudent? s = await
      ↪  transaction.get.one<ComputingStudent>(student.id);
      if (s != null) {
        student.firstname = "CHANGED!";
        transaction.update.one(student);
      }
    }).then((value) {
      FS.get.one<ComputingStudent>(student.id).then((result) {
        assert(result != null);
        if (result != null) {
          assert(result.firstname == "CHANGED!");
        }
      },);
    },);
});
```

Code 3. An integration test, testing the behavior of object-oriented transactions in Firestore.

```
testWidgets("Test list.ofClass() without limit", (tester)
↪  async {
    List<ComputingStudent> result = await
    ↪  RDB.list.ofClass(ComputingStudent);
    assert(result.length == students.length);
});

testWidgets("Test list.ofClass() with limit", (tester)
↪  async {
    List<ComputingStudent> result = await
    ↪  RDB.list.ofClass(ComputingStudent, limit: 3);
    assert(result.length <= 3);
});
```

Code 4. An integration test, testing the behavior of `RDB.list.ofClass()`.

Integration tests were useful in identifying a plethora of issues which went undetected during the development process. For instance, integration testing resulted to the identification of a serious bug in transaction logic, which arose from the fact that multiple `transaction` objects were being utilized within the code. Firebase's databases, unlike others, do not use lock-based transactions and instead rely on serialization-based transactions. In this approach, transactions do not lock resources on the database, but rather utilize document timestamps to track if a document has changed since it was last accessed by the transaction. Hence, each transaction can be idempotent, but only within the scope of the transaction itself and not for other transactions. Such a profound observation would not have been possible had it not been for integration tests which attempted to test the behavior of object-oriented transaction operations supported by Firestorm. Various other integration tests also helped with the verification of the behavior or several operations, ultimately leading to increased confidence in the consistency of the package's operations and its overall usefulness. Source code for all integration tests is available on GitHub under the test platform project [3].

4.4. Compatibility Testing

The aim of this project is to provide Firestorm for use in various major platforms, including Android, iOS, Web, and Windows (Firestore-only), and MacOS to a lesser extent. Several projects were employed to confirm whether the package functions as expected on these platforms. The testing platform which was used during development confirmed that the package works well on Android, whereas two other case study projects were used to confirm functionality on iOS, Web, and Windows. Testing on MacOS systems is still pending, and is considered a tertiary feature of the package.

[3]Integration tests: https://github.com/RayLabz/Firestorm/tree/master/flutter_test_app/integration_test

5. EVALUATION

5.1. Overview

The evaluation of Firestorm entails three different components to ensure a more general assessment of its effectiveness and suitability for development. The following sections describe each of these components and discuss the strategies used to obtain measurements and answer research questions.

5.2. Performance

The performance evaluation focuses on measuring the overheads introduced by Firestorm's object-oriented API, compared to the native non-object-oriented Firestore and Realtime database API operations. A benchmarking process aims to record the *processing time* of matching types of operations executed both using the Firestorm and Native APIs. This can be achieved by running code within the demonstrator app, which can run within an Android Emulator. The app can be connected to a running Firebase Emulator suite, that will be hosting only one of the two datastores at a time to avoid any performance degradation. The host device will feature an Intel i7 11700K processor with 32GB of RAM and an NVMe SSD with read and write speeds exceeding 5000MB/s. The results obtained from this experiment will be recorded in milliseconds, will be categorized for each operation, tabulated, and analyzed to identify any performance degradation or improvement compared to the native APIs. A *performance degradation threshold* is set to -10% of the native API performance. Performance that is worse than this threshold will be regarded as unsatisfactory for an operation, which means that the implementation of that operation must be reviewed at a later stage. This form of relative measurements ensures that the impact of any secondary factors is negated. Such factors include the performance characteristics of the host machine, the capacity and performance of the Android and Firebase emulators, network speed and throughput, processing spikes on the host machine, performance degradation due to Flutter's debugging mechanisms, and more, in line with recommendations from previous studies [33]. Other experimental features are also considered, such as the use of a common data model across all operations to ensure a consistent data size, the creation of nearly identical code with the same structure and clauses, and the use of multiple trials as part of the same experiment to increase the reliability of the data.

The performance of the code is determined by measuring the processing time in milliseconds. This is defined as the time taken for a specific set of statements specifically related to the operation being tested to be executed, received, and persisted by the database, and a response to be received at the client-side. In this evaluation only the main operations are compared, such as creating, retrieving, updating, and deleting items, along with their multi-object counterparts. This is done for both the Firestore and Realtime database APIs for Firestorm and the native SDKs correspondingly. In detail, the following operations are evaluated:

- Single object create (`FS.create.one()` and `RDB.create.one()`).
- Single object get (`FS.get.one()` and `RDB.get.one()`).
- Single object update (`FS.update.one()` and `RDB.update.one()`).
- Single object delete (`FS.delete.one()` and `RDB.delete.one()`).
- Multi-object create (`FS.create.many()` and `RDB.create.many()`).
- Multi-object get (`FS.get.many()` and `RDB.get.many()`).
- Multi-object update (`FS.update.many()` and `RDB.update.many()`).
- Multi-object delete (`FS.delete.many()` and `RDB.delete.many()`).

5.2.1. Measurement procedure

The processing time for each operation is measured using Dart's `Stopwatch` tool, which is a high-resolution timer. For each of the operations listed above, three trials are executed and averages are calculated to improve the reliability of the results. For instance, in single-object operations, three different trials are executed, each consisting of 100 synchronously executed reads (one after each other). To ensure this, the calls to these operations are made synchronously using `await`, and a delay of 100 milliseconds is added between operations as a padding to allow adequate time for Firestore to assimilate data and return to an idle state. For multi-object operations, each trial consists of just a single operation which operates on 100 objects, with three trials recording the time taken for this operation to be completed. The trials described above are meant to measure the processing time of these operations with a constant set of objects. During each run, the code automatically instantiates only the necessary data needed in these operations, which as subsequently deleted from the database after each test to avoid inconsistencies due to database load. The code for this setup is available on the project's GitHub repository [4].

5.2.2. Firestore results

Figure 6 compares the average processing time of single object operations in Firestore between the native API and Firestorm. As the results show, Firestorm's `FS.create.one()` operation marginally outperforms its Native API counterpart, whereas the rest of the single-object operations perform marginally worse than the Native API with `FS.update.one()` performing relatively worse with about 3% added processing time compared to the native API. However, the performance of all single-object operations in Firestore are well within the threshold margin of +10% increase in processing time.

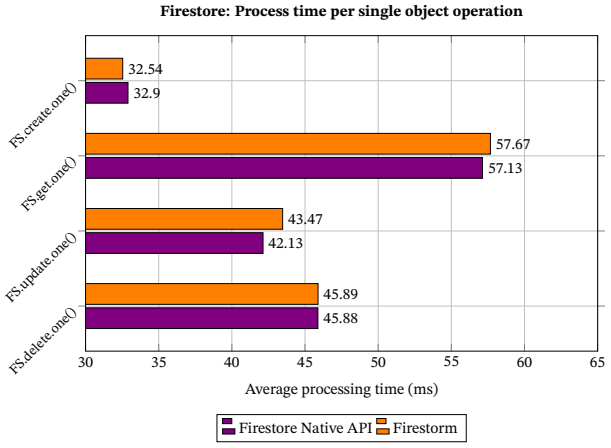

Figure 6. Average processing times for single-object operations in Firestore Native API and Firestorm

The results for multi-object operations are shown in Figure 7. From these, we can see that the `FS.create.many()` operation offers substantially better performance compared to its native API counterpart, with 17% less processing time. Meanwhile, the `FS.get.many()` and `FS.delete.many()` methods are on-par with their native API counterparts in terms of processing time and perform better by an insignificant amount. More interestingly, the `FS.update.many()` method seems to provide a significant improvement of 95% in processing speed over its corresponding Firestore API call. While the reasons for this are not yet fully understood, evidence from single-object operations suggests that update operations are overall more efficient than other types of operations in Firestore. As such, this method may benefit from taking advantage of multi-object write batches that are significantly more efficient at larger object counts but this remains open for further exploration.

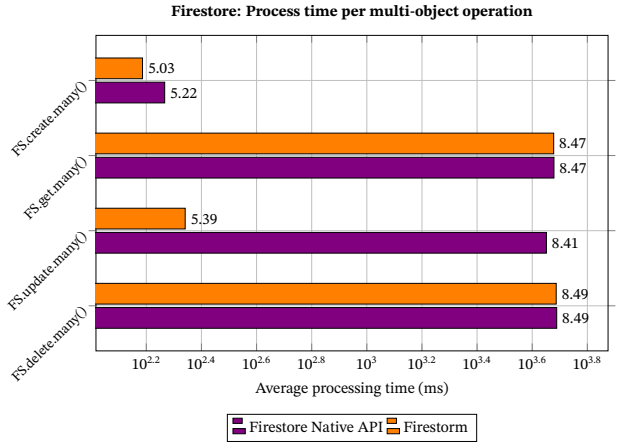

Figure 7. Average processing times for multi-object operations in Firestore Native API and Firestorm

5.2.3. Realtime database results

The Realtime database operations are also evaluated in an identical way. The average processing times for each single-object operation for the Realtime database are shown in Figure 8. The results show that the `RDB.create.one()` and `RDB.update.one()` operations introduce a 5.6% and 1.5% overhead on processing time correspondingly. At the same time the `RDB.delete.one()` method makes an insignificant performance impact, whereas `RDB.get.one()` enjoys a slight performance improvement over its native API counterpart. All methods, including the single-object create one which introduces some overhead, are below the maximum overhead threshold of +10%.

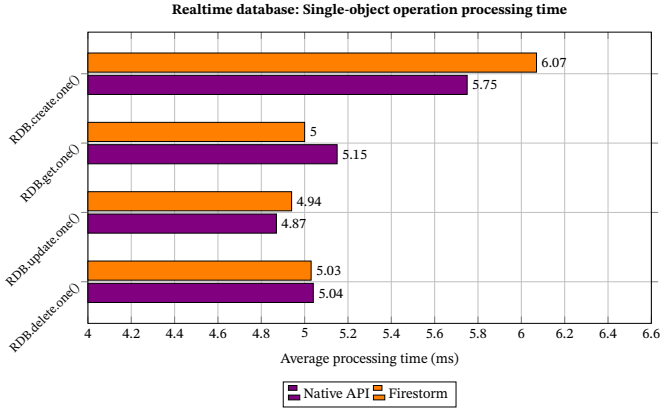

Figure 8. Average processing times for single-object operations in RDB Native API and Firestorm

Finally, multi-object operations in Realtime database were also tested. The results for these are shown in Figure 9. These provide evidence that Firestorm offers significant advantages for multi-object operations in Realtime database. The `RDB.create.many()`, `RDB.update.many()`, and `RDB.delete.many()` functions all enjoy a major performance improvement ranging from 84% to 92% compared to the native API functions. At the same time, the `RDB.get.many()` operation is on-par with the corresponding operation in the native API, suffering from a negligible performance overhead. With regards to multi-object operations in Realtime database, this data paints a clear picture: Firestorm not only stays below the performance overhead threshold, but also provides significant improvements over the native Realtime database API.

[4] Evaluation code: https://github.com/RayLabz/Firestorm/tree/master/flutter_test_app/lib/evaluation

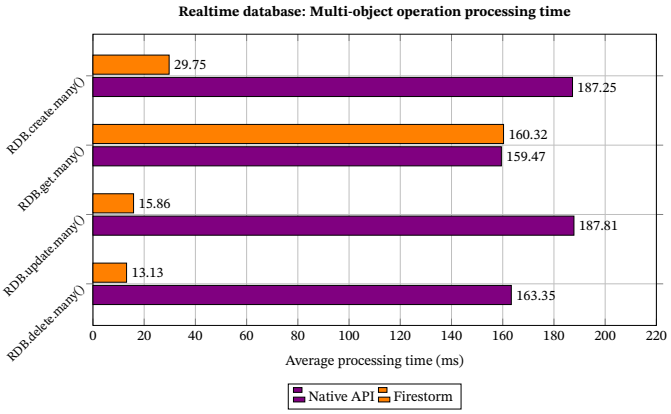

Figure 9. Average processing times for multi-object operations in RDB Native API and Firestorm

5.2.4. *Discussion on performance implications*

The results obtained from the performance evaluation to understand the performance overheads introduced by Firestorm compared to the Firebase SDKs, in terms of processing time. The results are analyzed in two ways. In the first, the percentage difference in performance is used to categorize each operation's overhead status into one of the following categories, using the following boundaries:

- **Significant overhead**: Above +10% increase in processing time
- **Slight overhead**: Between +10% and +5% difference in processing time
- **Negligible overhead**: Between +5% and >0% difference in processing time
- **Parity**: No (0%) difference in processing time
- **Negligible improvement**: Up to -5% decrease in processing time
- **Slight improvement**: Between -5% and -10% decrease in processing time
- **Significant improvement**: Over -10% decrease in processing time

Figure 10 shows the number of operations per category for Firestore. Three of these operations had negligible performance overhead, another three operations had negligible improvement, and two of operations made significant improvements in performance. This shows that for most operations in Firestore, the overhead or improvement in terms of performance was negligible, and likely subject to statistical error.

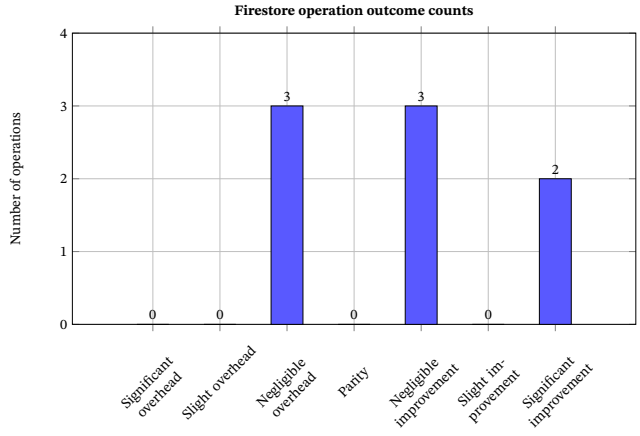

Figure 10. Firestore: Count of operations per performance category.

Figure 11 shows the same distribution per category for the Realtime database. In this case, three operations enjoyed significant improvement in performance, two enjoyed a negligible improvement, another two had a negligible overhead, and one suffered from a slight overhead. This data points out that the majority (5 out of 8) of operations enjoyed a performance improvement, whereas the rest of them suffered from a performance overhead. Importantly, the category with the most frequency in this case is the one of significant improvement, highlighting the fact that Firestorm provides a significant advantage in performance.

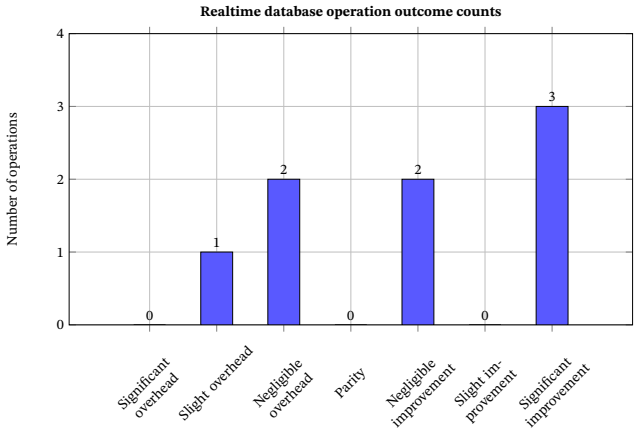

Figure 11. Realtime database: Count of operations per performance category

For both datastores combined, we can see that 5 operations had significant improvement, 5 had negligible improvement, 5 had negligible overhead, and 1 had a slight overhead. Based on this, most operations (10 out of 16) provided an improvement in performance, whereas 6 of them suffered from overhead.

The second way used to analyze the data is by averaging the differences between Native APIs and Firestorm among all operations. For the Firestore, this difference averages at -13.7%, which indicates a performance increase overall. The average difference is even higher for the Realtime database at -32.9%, indicating an even stronger performance increase. Combined, the average difference across all operations, across both datastores is -23.3%, which is a very positive outcome given that Firestorm adds new layers of software on top of those used to interact with Firebase.

The presence of so many operations with improvement in performance is surprising, as the expectation was that there would be at least a slight overhead in nearly all of the operations tested. However, the design strategies used to build Firestorm as well as programming techniques employed to reduce such overheads may have played a significant role in improving performance. Despite these positive results, the answer to the research question posed depends on the identification of the methods which did suffer from performance overheads. Based on this data, the worst performing operation was Realtime database's single object write, which is perhaps one of the most used operations. Despite this being only a slight overhead, the frequency of use of single object writes means that it is important to revisit the implementation of this method to identify ways to reduce these overheads if possible. Other methods, such as Firestore's single object retrieval, update, and delete are considered far less important, as their difference in performance is very close to 0, and statistically insignificant.

5.3. Code Maintainability

The second component of the evaluation is an assessment of the maintainability of the code when using Firestorm operations. To achieve this the demonstrator app will be used as a platform where groups of code snippets can be written. In each group of code snippets, one datastore operation will be tested, broken into two or three implementations — one for the Firestorm API and one (or two where applicable)

for the Native APIs for Firestore and Realtime database. As in the performance component, the code will utilize a single data entity to ensure consistency, and the code written will utilize identical code constructs. Various CRUD operations, batch operations, but also more advanced features such as batch writes, transactions, and real-time listeners will be tested. The maintainability of each code snippet will be measured using the Maintainability Index (MI) metric [34], which composes of (a) the Lines of Source Code (SLOC), (b) Halstead Volume (HV), and (c) Cyclomatic complexity (CC). The formula used to determine the Maintainability Index of a snippet of code is shown in Equation 5.3.

$$\begin{aligned}\text{Maintainability Index} = 171 \\ - 5.2 \times \ln(\text{Halstead Volume}) \\ - 0.23 \times \text{Cyclomatic Complexity} \\ - 16.2 \times \ln(\text{Lines of Code})\end{aligned} \tag{1}$$

Figure 12. Maintainability Index formula.

To measure Halstead Volume, Cyclomatic Complexity, and Lines of Code, the experiment will involve the use of an AI agent which conducts code analysis. The agent will receive code snippets as text-based input, calculate code metrics such as HV, CC, and SLOC while also reporting the process followed (i.e., tokens/paths considered, etc.). The output will report the code metrics discussed above, thereby enabling their tabulation and analysis. Based on the data gathered, a quantitative analysis can lead to conclusions on whether using Firestorm improves or worsens code maintainability compared to the native APIs, and whether it has the potential to simplify the code maintenance process.

5.3.1. Measurement procedure

Similar to the performance evaluation, the experimental setup makes use of the testing platform to host testing code. For this evaluation, a set of selected operations for Firestore was chosen as a sample. There are two reasons why only Firestore operations were chosen for this aspect of the evaluation. Firstly, the data access APIs of Firestore and the Realtime database are nearly identical (Firestore API is a superset of the Realtime database API), and secondly, the Firestore API enables more functionality to be tested, such as pagination and transactions, which are not supported by the Realtime database. Each operation was implemented in two snippets: one using the Firestore native API and one using Firestorm's API. As both native and Firestorm APIs support both blocking and non-blocking modes, both of these modes were tested for each operation selected. Where needed, versions of these functions were also tested using error checking and without error checking.

The code implemented in these snippets is syntactically correct but not necessarily compilable. The purpose of this evaluation is not to run the code, but simply measure the maintainability index of the code through code analysis. This is done by using an LLM-powered (Large Language Model) AI agent approach, using ChatGPT's o4-mini model. Firstly, the model's ability to identify code tokens and constructs in the Dart programming language was confirmed by dry-running a series of snippets to confirm the validity of the results (i.e., confirm the validity of weights attached to certain constructs, function calls, etc.). After this, the model was given one snippet of code at a time and was asked to count the non-empty lines of code, determine the cyclomatic complexity, calculate and Halstead volume of the snippet. Finally, the model tabulated the results, which were used to make comparisons between snippets written for the same operation. Listing 5 illustrates an example of two pieces of code written for the native and Firestorm API correspondingly. In this case, the pagination functionality is tested with a single page retrieval, in blocking mode, and without error checking in place. It should be noted that when using the native API, serialization and deserialization functions generated by Firestorm are used. In this case, it is assumed that developers would manually implement such functions themselves. While this provides an advantage to the native API approach, measuring the impact of producing this code for the native operations is beyond the scope of this test, although it is clear that undertaking such an activity would increase the overall time taken to write code, and the chances of the developers introducing errors. While this is important for maintainability, measuring the effects of this downside is beyond the scope of this evaluation.

```dart
//TEST PAGINATION_3 - Firestore API - Pagination, Blocking,
↪ no error checking
QuerySnapshot snapshot = await
↪ fs.collection('Student').orderBy('name').limit(10).get();
if (snapshot.docs.isNotEmpty) {
  List<ComputingStudent> students = snapshot.docs.map((doc)
  ↪ => ComputingStudent.fromMap(doc.data() as Map<String,
  ↪ dynamic>)).toList();
  print('Students retrieved successfully: $students');
} else {
  print('No students found');
}

//-------------------------------------------------------

//TEST PAGINATION_3 - Firestorm FS - Pagination, Blocking,
↪ no error checking
FSQueryResult<ComputingStudent> result = await
↪ FS.paginate<ComputingStudent>().next();
if (result.items.isNotEmpty) {
  print('Students retrieved successfully with Firestorm FS:
  ↪ ${result.items}');
} else {
  print('No students found with Firestorm FS');
}
```

Code 5. The pagination function used for MI analysis, in blocking mode, without error checking.

5.3.2. Results

The results presented in Table 1 are organized into test groups, where each test group contains multiple tests related to one operation, in blocking, non-blocking, and error handling versions. The results of these tests are averaged per test group to determine the overall MI for that operation, where higher MI is preferred.

Test group	Firestore API	Firestorm	Difference
CREATE_ONE	76	78	2.56%
CREATE_MANY	67.667	78.333	13.62%
DELETE_ONE	76	78	2.56%
GET_ONE	58.667	75.667	22.47%
UPDATE_ONE	76	78	2.56%
PAGINATION	54.333	61.667	11.89%
QUERY	50.667	56.667	10.59%
REAL_TIME_LISTENERS	60	66	9.09%
TRANSACTION	52	59	11.86%

Table 1. Comparison of Maintainability Index for the Firestore API and Firestorm operation.

For three of the single-object operations (create, delete, update), the maintainability indices of the Firestore API and Firestorm are identical at 76 and 78 respectively, indicating a slight maintainability improvement of 2.56% in each. For the single-object retrieval operation group (GET_ONE), this improvement is much more significant, at 22.47%. In the multi-object operation group (CREATE_MANY), the Firestore API scored a 67.7 MI, whereas Firestorm scored 78.3 — a significant increase in MI (13.62%). Several more advanced operations were also tested, such as Pagination, Queries, Real-time listeners, and Transactions. For pagination and transactions, the improvement in MI was about 12%, in queries about 10.5%, and for real-time listeners this difference was slightly lower at about 9%. These test groups can be further categorized in single-op, multi-op, and complex-op meta-groups, with the purpose of determining the MI impact on these types of operations more generally. From this we can see that the

most significant improvement in MI stems from multi-op operations (average: 13.62%), followed by complex ops (average: 10.82%), with the single-op group impacted the least with an average MI difference of 7.43%. Among all operations tested, the average MI index difference between the Firestore API code and Firestorm code indicates that Firestorm code is 10.62% more maintainable compared to native API code.

5.3.3. Discussion

The results obtained from this experiment can be used to determine the extent at which using Firestorm can improve code maintainability in developed applications compared to using native Firebase SDKs. The data shows that Firestorm provides measurable improvements in maintainability across all of the operation categories. The single-object operations demonstrated slight gains, which suggests that even in simple scenarios where code complexity is limited, the abstraction layer introduced by Firestorm contributed positively to maintainability. The most significant impact was observed in multi-object operations which are slightly more complex, as well as in single-object retrievals. In these cases the improvement suggests that these abstractions were more effective, most likely due to reduced boilerplate code. The complex operations benefited notably as well, at approximately 10.82%. These results suggest that as operational complexity increases, the relative advantage of using Firestorm also increases. Overall, the results show that using Firestorm can improve maintainability by about 10%. This is a substantial difference considering that significantly more complex operations could be carried out in real-world scenarios. In addition, we must also take into consideration the fact that Firestorm also removes the cumbersome, tedious, and error-prone process of manually writing serialization/deserialization logic. These improvements in MI, combined with the removal of creating and upkeeping this aspect of code can be a critical determinant of the long-term evolution of a project.

5.4. Feature coverage and comparative analysis

The third component of the evaluation is a feature coverage comparative analysis. Various similar tools will be considered in multiple programming environments such as Dart, Java, NodeJS, and more. The analysis also identify popular ORM/ODM features, and will consider basic feature coverage, as well as more advanced features. The purpose of this analysis is twofold. Firstly, it aims to explore the features of other ODM tools already in use and enable a comparison between them and against Firestorm. Secondly, it aims to determine Firestorm's percentage of feature coverage, situate it within the broader ecosystem of ORM/ODM tools, and reveal its potential limitations.

5.4.1. Procedure

The process of identifying these tools utilized generative AI to conduct a search on the web. A list of popular ODM and ORM tools were identified by instructing ChatGPT to look online such tools using the following prompt: "Search the web to make a list of ODM and ORM tools in various frameworks". The initial response identified a total of 16 tools across 8 different frameworks. This list of tools was expanded with four additional known ORM tools which were not included: Sequelize (Node.js, SQL), Typesaurus (Flutter, Firestore), Hibernate (Java), and Objectify (Java, Cloud datastore). This resulted to a total number of 20 tools.

In the second step, ChatGPT was tasked to identify common features across ORM and ODM tools with the following prompt: "Search the web to identify common features across ORM and ODM tools". This led to a response containing a list of 8 features, including schema enforcement, support for asynchronous operations, and more. Like in the previous step, the list of features was expanded with 5 additional features from related works and knowledge of existing tools: transactions, real-time listeners, pagination, migration support, and aggregation query support. The final number of features explored is 13.

ChatGPT was then asked to construct a feature matrix structure of these tools and the features they support with this prompt: "Make a matrix of these tools (as rows) and the features they support (as columns). Look on the web and make sure you find justifiable evidence online for the inclusion of such features before you confirm whether they are present in each framework or not. For each of the tools, include the resources you used to find this information as links". This response constructed a feature matrix by using 53 resources from the web. In the matrix, each supported feature is represented by a note within the cell indicating the presence of the feature.

Due to its size, the resulting matrix had to be broken into four smaller tables. In the first table, the database supported by each tool, schema enforcement support, and type safety features were included. The second table includes core capabilities such as query building, data validation, etc. whereas the third table includes data relationship and more advanced features such as listeners and pagination. The final table includes more advanced features such as aggregation, transactions, and more. Using the data provided in these tables, (a) the number of tools supporting a specific feature can be extracted, which can provide insights into which features are more popular, and (b) Firestorm's feature support can be compared with other tools to determine its feature coverage and determine areas where further work might be needed to address additional requirements.

Feature	Number of Tools (/20)	Firestorm
Schema Enforcement	19	Yes
Type Safety	19	Yes
Maintained	19	Yes
Query Builder	17	Yes
Validation	15	Yes
Pagination	14	Yes
Relationships	14	
Hooks	12	Yes
Aggregation	12	
Async Support	10	Yes
Transactions	11	Yes
Realtime Listeners	7	Yes
Migration	2	

Table 2. Frequency of feature support across all tools.

5.4.2. Results

The results are summarized in Table 2 which presents these features sorted by frequency of tools supporting these features. The third column in the table indicates the presence of each feature in Firestorm, allowing an evaluation of the feature coverage.

Table 3 also shows the number of features supported by each tool, sorted in descending order. This can provide a more in-depth view of each tool's feature coverage, allowing direct comparisons between these tools.

5.4.3. Discussion

The results shown in Table 2 reveal a high level of diversity in terms of feature support in the ORM/ODM tools studied. Feature adoption is relatively high, with most of the features being supported by the majority of the tools. The schema enforcement, type safety, and maintenance were the most popular aspects of these packages, with all but one tool supporting these features. Other strongly supported features were query building support, validation mechanisms, pagination, and relational constructs between data. Unsurprisingly, all of these features are often regarded as cornerstone requirements of such tools. Other features which are typically less supported include aggregation queries, support for asynchronous programming, transactions, and real-time listeners, whereas migration is mostly ignored.

Table 3 lists the number of features supported by each individual tool.

Tool	Number of Features Supported (/13)
Mongoose	11
Sequelize	11
Spring Data Mongo	11
Typegoose	11
Firestorm	10
Hibernate	10
Mongoid	10
Objection.js	10
Beanie	10
cloud_firestore_odm	9
MongoDB .NET Driver	9
MongoEngine	9
Morphia	9
Objectify	9
mongo-go-driver	8
JNoSQL	7
Laravel MongoDB	6
Waterline	6
Marshmallow	5
Isar	4
Pydantic ODM	4

Table 3. Number of supported features per tool.

8 out of 20 (40%) of these tools had excellent (more than 75%) support for the features identified, and 13 out of 20 (65%) provided a support for the majority (at least 50%) of the features. The data in these two tables can be used to determine how Firestorm compares against other ODM tools in terms of features. As seen in table 2, Firestorm provides support for 10 out of 13 (≈77%) of the features identified, which provides a very good level of feature coverage. This places it near the top of Table 3, which counts the number of features supported in each tool. Thus, in terms of feature coverage, Firestorm outperforms at least 12 other popular ORM/ODM tools, and it is only outperformed by 4. The results obtained from this aspect of the evaluation can also be very useful in determining new directions in development. For instance, next steps in development could focus on features which are currently unsupported or undersupported, such as the formation of relationships between data and support for aggregation queries.

5.4.4. Design strategies making Firestorm successful against other tools

A major difference between Firestorm and most of the tools explored in the comparative feature analysis is that most of these tools focus on providing ORM/ODM support for a single type of database. While some tools such as Marshmallow provide database and framework-agnostic support for various features, these focus on data serialization and deserialization from/to platform-agnostic representations such as JSON rather than implementing specific data access APIs. An exception is Java's Hibernate, which uses the `EntityManager` and JPA (Java Persistence API) to address the problem of managing data interactions regardless of the database used. At present, the Flutter ecosystem does not have a production-grade ORM/ODM tool that offers this level of abstraction. Firestorm is built based on several architectural principles and design strategies to address this problem:

1. **Multilayered modular architecture**
   The architecture employed by Firestorm described in section 3.5 promotes the separation of concerns between different aspects of the app including data modeling, code generation, and the data access components. Firstly, this leads to a better level of maintainability across the package. More importantly however, this introduces an abstraction layer, which is present within the data access API that enables a common API to be utilized for both datastores, and any future datastore that needs to be supported.
2. **Delegate pattern**
   An important design aspect of Firestorm is the use of the Delegate pattern to encapsulate datastore-specific CRUDLE operations. The delegates implement abstract API operations, but also further decouple code and promote abstraction and concern separation at a more granular level.
3. **Facade pattern**
   The data access API exposes unified entry points for each datastore – the `FS` and `RDB` classes. While these are independent of each other, the use of the Facade design pattern in combination with Delegates provides a *consistent* API for both datastores and enables developers to use them simultaneously.
4. **Static analysis and Code generation**
   Runtime reflection, a performance-degrading and unsupported feature in Flutter, has been replaced by compile-time code analysis and generation to ensure type safety and consistent serialization and deserialization. This mechanism is common across both datastores, but still handles the data type support peculiarities of each specific datastore.
5. **Registry & Strategy patterns**
   Firestorm utilizes the Registry and Strategy patterns to dynamically map entity metadata and associated functions such as serializers and deserializers, enabling it to efficiently use statically defined data at runtime to perform class validation.
6. **Builder pattern**
   The use of builders promotes better code structures by using the declarative programming paradigm. Using this pattern, developers can easily define complex queries that are compatible with each datastore.

6. CONCLUSION

This project addressed the lack of mature, maintainable, and developer-friendly ODM tools to facilitate data management in cross-platform Flutter applications. It presented Firestorm, an ODM and data access API which facilitates maintainable cross-platform software development, its architecture, design components, testing platform, and evaluation with respect to three aspects – performance, code maintainability, and feature coverage.

6.1. Contributions

6.1.1. Summary of objectives

The project's objectives which were outlined in section 1.2 included – among others – the reduction of repetitive boilerplate code, the enhancement of code maintainability, the preservation of performance, and provisions for supporting multiple datastores simultaneously.

1. **Declarative & type-safe data modeling**
   Various components in this package utilize a declarative approach to make it easier to define both data models and queries. Code analysis and generation are used to enforce type safety and a consistent but flexible schema, which reduces the likelihood of runtime errors but also preserves the ability of software to evolve.
2. **Reduction of boilerplate code**
   Firestorm automates the logic of mapping objects to their serialized counterparts and back which significantly reduces the effort and time needed to develop and expand applications. This also prevents developers from introducing bugs within the mapping/serialization pipeline.
3. **Facilitating RAD**
   Features such as automatic code generation and high-level abstractions in multiple layers of Firestorm are considered to act as a conduit towards Rapid Application Development. Using Firestorm, developers can quickly prototype applications that must utilize Firebase's datastores for persistence with an easy-to-use, low-code, object-oriented API. This can enable the rapid prototyping of applications in educational and research settings, but also in commercial contexts where the time-to-market is more important.
4. **Improved code readability, maintainability, and portability**
   Firestorm's object-oriented approach is inherently more readable

than attribute-oriented code utilized in the native Firebase APIs, which makes it easier to read and understand. Evidence has also suggested that code maintainability is increased, especially for larger projects which typically rely on such backend services. Furthermore, the nature of Flutter as a cross-platform framework means that Firestorm's code is inherently portable.

5. **A unified API for multiple datastores**
   Firestorm provides a single, intuitive data access API which allows the development of applications that interact with multiple datastores at the same time. Even though this project focuses on Firebase's Firestore and Realtime databases, it could be easily expanded to use the same API to support other types of databases.

6.1.2. Summary of research output

The evaluation assessed the package's overall effectiveness with respect to the evaluation aspects identified in section 1. The maintainability index analysis has shown code maintainability improvements across all of the tested operations – even among relatively simple CRUDLE operations where the code is already simple. While not covering all of the operations, the sample of operations tested presents a representative sample of various types of operations typically used by developers to manage data. Thus, it is possible to claim that code written using Firestorm has the potential of improving code maintainability by an average of 10% compared to using the native Firebase Flutter database SDKs.
Another research output of this project is presented in section 5, which presents a list of 6 design strategies used to enable Firestorm to support both Firestore and the Realtime database within a single ODM tool. Firestorm's ability to handle simultaneous operations across both datastores provides proof that these design strategies have successfully enabled Firestorm to meet its objectives. Such design strategies not only simplify development but also provide the underpinnings for seamless interoperability between datastores.
Performance-wise, the results obtained from running isolated data access API code tests show that Firestorm not only stays within the acceptable 10% processing time increase threshold, but surprisingly outperforms the native APIs in several operations. In some of these operations, significant performance gains were also observed, highlighting the success of Firestorm as a performance-sensitive data access API that allows efficient operations despite the added layers of processing needed to enable ODM.
In terms of feature coverage, the comparative analysis conducted indicates that Firestorm implements the majority (77%) of the features typically expected in a mature ORM/ODM framework, and also exceeds or ties most (80%) of the identified tools in terms of feature coverage.

6.2. Limitations and future work

Despite these very promising results, there are several limitations in both the development and testing of Firestorm, as well as the methods and processes used to evaluate it. These are discussed below, along with plans to mitigate these limitations through future work.

6.2.1. Limitations in Development and Testing

Firstly, the package is currently missing several features which might be useful to developers such as data relationships and aggregation queries. While such features are not part of v1, the package's underlying architecture enables the expansion of its APIs to easily implement these capabilities in the future.
Secondly, the project has limited coverage in terms of unit, integration, and compatibility testing. Despite testing and verifying Firestorm within various environments, there is limited proof that this package is compatible with MacOS. While an analysis of the internal dependencies shows support for this platform, further platform compatibility testing is required.
Thirdly, while Firestorm is designed to enforce type safety and validation operations at runtime, there could be further issues which might have escaped testing. For instance, runtime configuration errors or particular edge cases during serialization/deserialization could still introduce hard-to-detect bugs in the package which would only be detected through its usage (over time), or through exhaustive testing. This is typical for cases where automated unit tests cannot be performed due to running compile-time tasks. Therefore, future work should focus on maintaining the package by tracking issues that arise and fixing the underlying causes to improve its quality, and by improving the test coverage.

6.2.2. Limitations in the Research approach

Despite successfully addressing several research questions, the research approach used may limit the generalizability of the results presented in this project.
The package's performance benchmarking was conducted in a controlled environment to ensure the reproducibility and reliability of the results. While this ensures reproducibility, it also limits the generalizability of the results. For instance, real-world deployments involving the actual datastores (not emulators), diverse network conditions, varying device capabilities, and production-scale data volumes could lead to dramatically different performance characteristics. Future work can focus on simulating real-world, production-scale data requirements, more diverse conditions in varying usage contexts, as well as measuring the memory overheads.
Furthermore, the maintainability analysis based on the Maintainability Index metric uses pre-defined snippets of code from the native APIs and Firestorm. Despite the fact that MI is a composite metric which captures three different dimensions of maintainability, it still does not capture all dimensions. For instance, MI does not provide insights with regards to developer perceptions, the learnability of the code, or the long-term maintainability under use by large groups of developers.
The comparative analysis of feature coverage relied on publicly available information obtained from the web with the help of a generative AI tool. While the documentation and online resources identified were verified as authentic, there are concerns over the accuracy of the results produced by an AI tool, especially when it comes to identifying the right features and tools. A more exhaustive search could be necessary to identify tools, features, and resources which may have been overlooked.

6.3. Conclusion

Despite these limitations, Firestorm has the potential to significantly improve the maintainability of cross-platform software which incorporates data-driven operations. The project has completed most of the requirements identified and has presented results which have demonstrated significant advantages in terms of the evaluation aspects. This package is now publicly available and open-source, and the author is hopeful that it may not only improve the process of developing cross-platform software, but also inspire further research in this domain.

REFERENCES

[1] M. Yousuff, V. Anusha, J. Vijayashree, and J. Jayashree, "The role of artificial intelligence for intelligent mobile apps," in *Emerging Technologies for Sustainable and Smart Energy*. CRC Press, 2023, pp. 185–205.

[2] Y. Yazid, I. Ez-Zazi, A. Guerrero-González, A. El Oualkadi, and M. Arioua, "Uav-enabled mobile edge-computing for iot based on ai: A comprehensive review," *Drones*, vol. 5, no. 4, p. 148, 2021.

[3] H.-P. Halvorsen, "Introduction to database systems," *Telemark University College, dated Mar*, vol. 3, p. 40, 2014.

[4] D. Maier, *The theory of relational databases*. Computer science press Rockville, 1983, vol. 11.

[5] P.-J. Liu, C.-P. Li, and H. Chen, "Enhancing storage efficiency and performance: A survey of data partitioning techniques," *Journal of Computer Science and Technology*, vol. 39, no. 2, pp. 346–368, 2024.

[6] A. S. Shethiya, "Load balancing and database sharding strategies in sql server for large-scale web applications," *Journal of Selected Topics in Academic Research*, vol. 1, no. 1, 2025.

[7] S. Ponnusamy and P. Gupta, "Scalable data partitioning techniques for distributed data processing in cloud environments: A review," *IEEE Access*, vol. 12, pp. 26 735–26 746, 2024.

[8] A. Ravindran and A. George, "An edge datastore architecture for {Latency-Critical} distributed machine vision applications," in *USENIX workshop on hot topics in edge computing (HotEdge 18)*, 2018.

[9] Z. Liao, R. Zhang, S. He, D. Zeng, J. Wang, and H.-J. Kim, "Deep learning-based data storage for low latency in data center networks," *IEEE Access*, vol. 7, pp. 26 411–26 417, 2019.

[10] S. H. Kamal, H. H. Elazhary, and E. E. Hassanein, "A qualitative comparison of nosql data stores," *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 2, 2019.

[11] L. Mathiassen, A. Munk-Madsen, P. A. Nielsen, and J. Stage, *Object-oriented analysis & design*. Citeseer, 2000, vol. 25.

[12] T. Karanikiotis and A. L. Symeonidis, "Towards understanding the impact of code modifications on software quality metrics," *arXiv preprint arXiv:2404.03953*, 2024.

[13] pub.dev, "firestore_odm | Flutter package — pub.dev," https://pub.dev/packages/firestore_odm, 2025, [Accessed 02-07-2025].

[14] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley, 2012.

[15] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, W. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles*. ACM, 2007, pp. 205–220.

[16] A. Lakshman and P. Malik, "Cassandra: a decentralized structured storage system," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 2, pp. 35–40, 2010.

[17] M. A. Rodriguez, "The graph traversal pattern," *Communications of the ACM*, vol. 58, no. 3, pp. 128–136, 2015.

[18] J. Pokorny, "Nosql databases: a step to database scalability in web environment," *International Journal of Web Information Systems*, vol. 9, no. 1, pp. 69–82, 2013.

[19] E. A. Lee, S. Bateni, S. Lin, M. Lohstroh, and C. Menard, "Quantifying and generalizing the cap theorem," *arXiv preprint arXiv:2109.07771*, 2021.

[20] A. Moniruzzaman and S. A. Hossain, "Nosql database: New era of databases for big data analytics-classification, characteristics and comparison," *International Journal of Database Theory and Application*, vol. 6, no. 4, pp. 1–14, 2013.

[21] L. Moroney, *The Definitive Guide to Firebase: Build Android Apps on Google's Mobile Platform*. Apress, 2017.

[22] A. Gajendran and S. Ghosh, "Scalability and performance analysis of firebase realtime database," *International Journal of Computer Applications*, vol. 162, no. 7, pp. 1–5, 2017.

[23] N. Kasenides and N. Paspallis, "Athlos: A framework for developing scalable mmog backends on commodity clouds," *Software*, vol. 1, no. 1, pp. 107–145, 2022.

[24] M. Pavlov *et al.*, "Serverless computing: Economic and architectural impact," *IEEE Cloud Computing*, vol. 3, no. 5, pp. 24–30, 2016.

[25] C. Zhang, J. Peng, C. Xu, Q. Xu, and C. Yang, "Mitigating the impedance mismatch between prediction query execution and database engine," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–28, 2025.

[26] S. Barker, "Building node.js and mongodb applications with mongoose," *IBM DeveloperWorks*, 2013, https://developer.ibm.com/articles/wa-nodejs-mongoose-app/.

[27] J. Han, E. Haihong, G. Le, and J. Du, "Survey on nosql databases," *IEEE International Conference on Pervasive Computing and Applications*, pp. 363–366, 2011.

[28] N. Kasenides, "Models, methods, and tools for developing mmog backends on commodity clouds," Ph.D. dissertation, UCLan Cyprus, 2023.

[29] N. Kasenides, A. Piki, and N. Paspallis, "Exploring the user experience and effectiveness of mobile game-based learning in higher education," in *International Conference on Human-Computer Interaction*. Springer, 2023, pp. 72–91.

[30] D. Schroeder, N. Paspallis, N. Kasenides, K. Chatfield, C. Seedall, and M. Singh, "The prepared case study app for deep reflection on research ethics challenges during crises," *Nature Medicine*, pp. 1–2, 2025.

[31] E. Hare and A. Kaplan, "Designing modular software: a case study in introductory statistics," *Journal of Computational and Graphical Statistics*, vol. 26, no. 3, pp. 493–500, 2017.

[32] B. B. Mayvan, A. Rasoolzadegan, and Z. G. Yazdi, "The state of the art on design patterns: A systematic mapping of the literature," *Journal of Systems and Software*, vol. 125, pp. 93–118, 2017.

[33] V. Reniers, D. Van Landuyt, and W. Joosen, "Object-nosql database mappers: A benchmark study on the performance overhead," in *Proceedings of the 12th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)*. SciTePress, 2017, pp. 141–149.

[34] T. Heričko and B. Šumak, "Exploring maintainability index variants for software maintainability measurement in object-oriented systems," *Applied Sciences*, vol. 13, no. 5, p. 2972, 2023.